

Stopwatch Design Using Verilog HDL

RTL Design, Simulation and Verification Report

By

Priya Nageswari Karanam

Table of Contents

1. Abstract	-	3
2. Introduction	-	3
3. Objective	-	3
4. Design Methodology	-	4
5. Verilog HDL Code	-	4
6. Testbench Code	-	6
7. Code Explanation	-	6
8. RTL Schematic	-	7
9. Simulation Results	-	8
(a) Vivado Simulation Output	-	9
(b) ModelSim Simulation Output	-	9
10. Future Scope	-	10
11. Conclusion	-	10

1 Abstract

This project presents the RTL design and functional verification of a stopwatch using Verilog HDL. The system is designed to perform controlled time counting through sequential counter architecture capable of displaying hours, minutes, and seconds.

The stopwatch incorporates dedicated start, stop, and reset controls to manage counting operations dynamically. A debounce mechanism is implemented to ensure stable response from push button inputs and to avoid unintended signal transitions caused by switch noise.

The design behavior is analyzed using simulation waveforms generated in Vivado and ModelSim. Through this implementation, the project demonstrates practical concepts related to sequential logic design, timing control, counter synchronization, and control-driven digital systems.

2 Introduction

Time measurement systems play an important role in modern digital electronics and embedded applications. Stopwatch circuits are widely utilized in environments where controlled timing, event tracking, and elapsed time monitoring are required.

The development of a stopwatch using Verilog HDL provides insight into the design of synchronous digital systems involving cascading counters, control signals, rollover conditions, and sequential state transitions. The interaction between external control inputs and timing logic forms an essential part of real-time digital circuit behavior.

This project implements a stopwatch architecture capable of pause and resume operations using dedicated start and stop controls. The design process also introduces stable input handling through debounce circuitry and demonstrates how simulation tools can be used to observe and verify sequential timing behavior.

3 Objective

- To design a stopwatch system using Verilog HDL based on sequential digital logic.
- To implement start, stop, and reset functionalities for controlled timing operations.
- To develop cascading counter architecture for displaying hours, minutes, and seconds.
- To incorporate debounce logic for stable push button signal detection.
- To verify the functional behavior of the stopwatch using Vivado and ModelSim simulation tools.
- To understand timing control, counter synchronization, and sequential circuit behavior through RTL implementation.

4 Design Methodology

The stopwatch design is developed using a modular sequential logic approach in Verilog HDL. The overall architecture consists of debounce modules, enable control logic, and cascading counters for hours, minutes, and seconds.

The debounce module is used to stabilize push button inputs and eliminate unwanted signal transitions caused by switch bouncing. Separate debounce circuits are implemented for both start and stop control signals.

An enable control mechanism is designed to manage stopwatch operation dynamically. When the start signal is detected, the stopwatch begins counting, and when the stop signal is activated, the current counter values are retained without resetting the system.

The counting mechanism is implemented using hierarchical counter logic. The seconds counter increments continuously and triggers minute incrementation after rollover conditions. Similarly, minute rollover increments the hour counter, enabling complete time progression functionality.

The entire design is verified using a dedicated Verilog testbench in Vivado and ModelSim to analyze start, stop, hold, reset, and rollover behavior through simulation waveforms.

5 Verilog HDL Code

```
module debounce(input clk,button, output reg clean_data);
    parameter max_count = 10;    //1000000 ten lakhs
    reg [21:0]counter = 0;
    initial clean_data = 0;
    always @ (posedge clk) begin
        if(button == clean_data)
            counter <= 0;
        else begin
            if(counter == max_count) begin
                clean_data <= button;
                counter <= 0;
            end
            else
                counter <= counter + 1;
        end
    end
endmodule

module stop_watch(input clk,rst,start,stop, output reg [3:0]h1,h0,m1,m0,s1,s0);
    wire p_start,p_stop;

    debounce d1(clk,start,p_start);
    debounce d2(clk,stop,p_stop);

    reg en = 0;
    always @ (posedge clk)begin
        if(p_start == 1)
```

```

        en <= 1;
    else if(p_stop == 1)
        en <= 0;
end
initial begin
    h0 <= 0; h1 <= 0; m0 <= 0; m1 <= 0; s0 <= 0; s1 <= 0;
end
always @(posedge clk) begin
    if(rst)
        begin h0 <= 0; h1 <= 0; m0 <= 0; m1 <= 0; s0 <= 0; s1 <= 0; end
    else if(en) begin
        if(h1 ==2 && h0 == 3 && m1 == 5 && m0 == 9 && s1 == 5 && s0 == 9) begin
            h0 <= 0; h1 <= 0; m0 <= 0; m1 <= 0; s0 <= 0; s1 <= 0; end
        else begin
            s0 <= s0 + 1;
            if(s0 == 9) begin
                s0 <= 0;
                s1 <= s1 + 1;
                if(s1 == 5) begin
                    s0 <= 0; s1 <= 0;
                    m0 <= m0 + 1;
                    if(m0 == 9)begin
                        s0 <= 0; s1 <= 0; m0 <= 0;
                        m1 <= m1 + 1;
                        if(m1 == 5) begin
                            m0 <= 0; m1 <= 0; s0 <= 0; s1 <= 0;
                            h0 <= h0 + 1;
                            if(h0 == 9 ) begin
                                m0 <= 0; m1 <= 0; s0 <= 0; s1 <= 0; h0 <= 0;
                                h1 <= h1 + 1;
                                if(h1 == 2 && h0 == 3) begin
                                    h0 <= 0; h1 <= 0; m0 <= 0; m1 <= 0; s0 <= 0; s1 <= 0;
                                end
                            end
                        end
                    end
                end
            end
        end
    end
end
end
end
end
end
end
end
endmodule

```

6 Testbench Code

```
module stop_watch_tb;
    reg clk , rst , start , stop;
    wire [3:0] h0,h1,m0,m1,s0,s1;

    stop_watch dut(clk,rst,start,stop,h1,h0,m1,m0,s1,s0);
    always #5 clk = ~clk;
    initial begin
        clk = 1; rst = 1; start = 0; stop = 0; #10;
        rst = 0; #20;

        start = 1; #150;
        start = 0; #2000;

        stop = 1; #150;
        stop = 0; #2000;

        start = 1; #150;
        start = 0; #2000;

        stop = 1; #150;
        stop = 0; #2000;

        start = 1; #150;
        start = 0; #1000;
        $stop;
    end
endmodule
```

7 Code Explanation

Debounce Logic

Mechanical push buttons often produce unstable signal transitions during pressing and releasing operations due to switch bouncing. The debounce module is implemented to eliminate these unwanted fluctuations and generate stable control signals.

The debounce circuit continuously monitors the button input and validates the signal only after it remains stable for a predefined number of clock cycles. Separate debounce modules are used for both start and stop inputs to ensure reliable stopwatch control operation.

Start and Stop Functionality

The stopwatch operation is controlled using dedicated start and stop signals. The debounced start signal activates the enable register and initiates stopwatch counting operation.

When the stop signal is detected, the enable signal is disabled, causing the stopwatch to pause while preserving the currently displayed time value. This enables hold and resume functionality without resetting the counter outputs.

Rollover Conditions

The stopwatch uses cascading counter architecture for seconds, minutes, and hours. The least significant counter increments first and generates rollover conditions for higher-order counters.

- The seconds counter increments continuously with each positive clock edge.
- When the seconds count reaches 59, the seconds counter resets to 00 and increments the minutes counter.
- When the minutes count reaches 59, the minutes counter resets to 00 and increments the hours counter.
- After reaching 23:59:59, the stopwatch automatically resets back to 00:00:00.

Verification Using Testbench

A dedicated Verilog testbench is developed to verify stopwatch functionality under different operating conditions. The simulation verifies reset operation, start and stop control behavior, counter hold functionality, restart operation, and rollover conditions through waveform analysis in Vivado and ModelSim.

8 RTL Schematic

The RTL schematic generated in Vivado represents the internal hardware structure of the stopwatch design. The schematic illustrates the connection between debounce modules, enable control logic, and cascading counter architecture used for time progression.

The RTL view clearly shows the interaction between start and stop control signals, sequential counter blocks, rollover logic, and output registers for hours, minutes, and seconds. The design primarily consists of synchronous sequential logic controlled through positive clock edge triggering.

The schematic also demonstrates the hierarchical flow of counter incrementation from seconds to minutes and hours along with reset and enable control behavior.

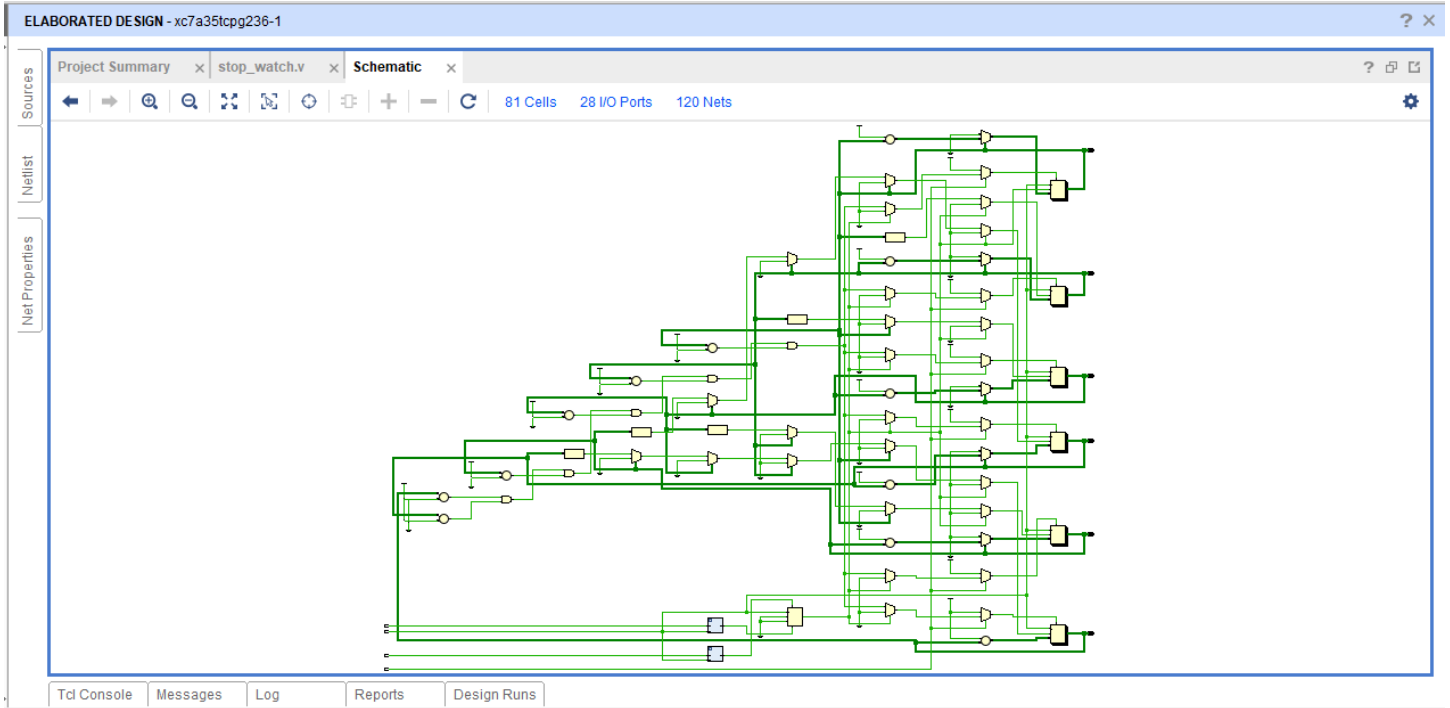


Figure: RTL Schematic of Stopwatch Design

9 Simulation Results

The functionality of the stopwatch design is verified using simulation in Vivado and ModelSim. The simulation process is used to analyze sequential timing behavior, enable control operation, rollover conditions, and counter synchronization.

Different operational scenarios including reset, start, stop, hold, and restart conditions are verified through waveform analysis using the Verilog testbench.

Vivado Simulation Output

The Vivado simulation waveform confirms correct stopwatch operation during start, stop, and reset conditions. The waveform also demonstrates proper cascading counter behavior for seconds, minutes, and hours along with enable-controlled timing progression.

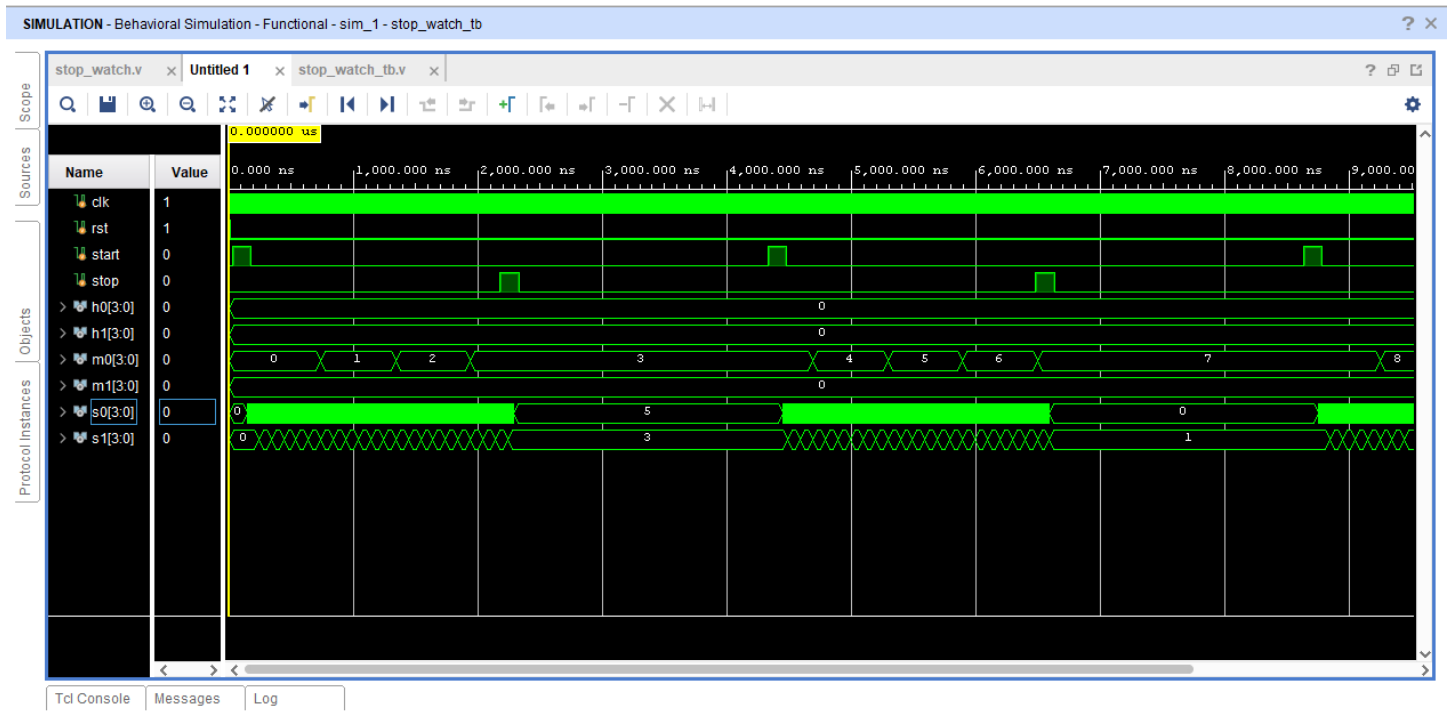


Figure: Vivado Simulation Waveform of Stopwatch Design

ModelSim Simulation Output

The ModelSim simulation output validates the functional correctness of the stopwatch design under different timing conditions. The waveform illustrates counter progression, rollover operation, stable control response, and hold functionality during stop conditions.

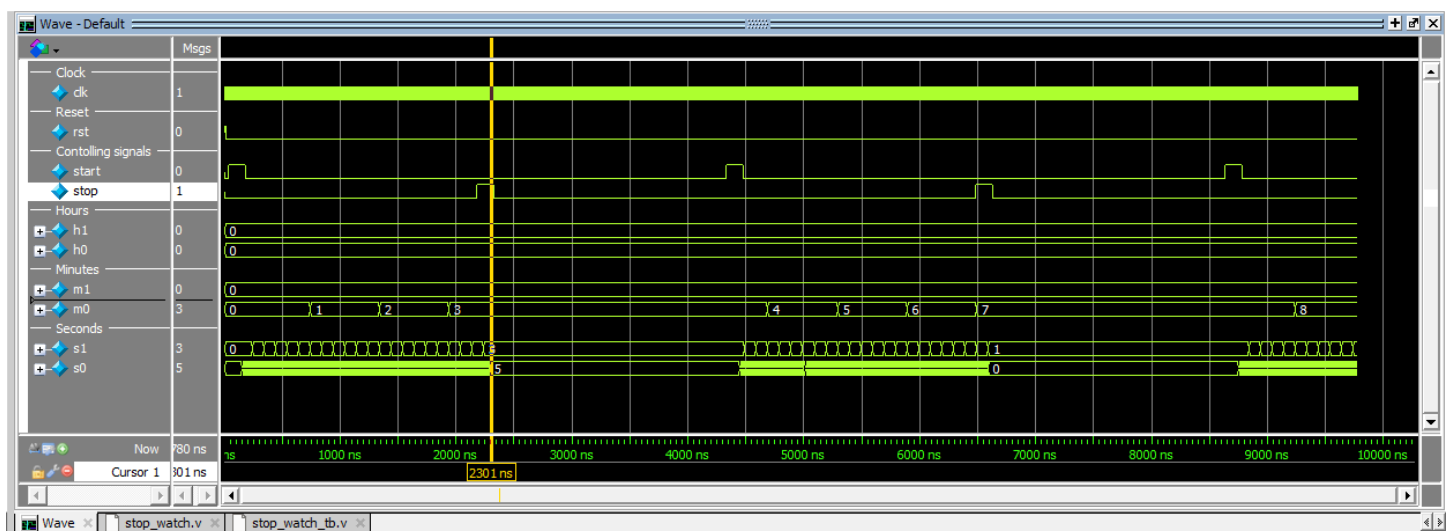


Figure: ModelSim Simulation Waveform of Stopwatch Design

10 Future Scope

FPGA Implementation

The stopwatch can be implemented on an FPGA board for real-time hardware operation. This helps in understanding practical digital system implementation.

Seven Segment Display

A seven-segment display can be connected to show stopwatch values externally. This improves the visual representation of hours, minutes, and seconds.

Lap Timing Feature

Additional features such as lap timing can be added to store intermediate timing values. This makes the stopwatch more useful for timing applications.

Improved Counter Design

The counter logic can be optimized to make the design simpler and more efficient. This can reduce hardware complexity and improve performance.

Better Debounce Logic

Advanced debounce techniques can be implemented for more stable button operation. This improves signal reliability during hardware execution.

Timer Applications

The stopwatch design can be extended to develop timer and countdown systems. This increases the practical applications of the project.

11 Conclusion

The stopwatch design using Verilog HDL was successfully implemented and verified through simulation. The project demonstrated the working of sequential digital logic using cascading counters for hours, minutes, and seconds.

The design successfully achieved start, stop, and reset functionalities along with stable input handling using debounce logic. Simulation results obtained from Vivado and ModelSim confirmed the correct operation of rollover conditions, control behavior, and counter synchronization.

This project provided practical understanding of RTL design, sequential circuit implementation, timing control, and simulation-based verification using Verilog HDL.